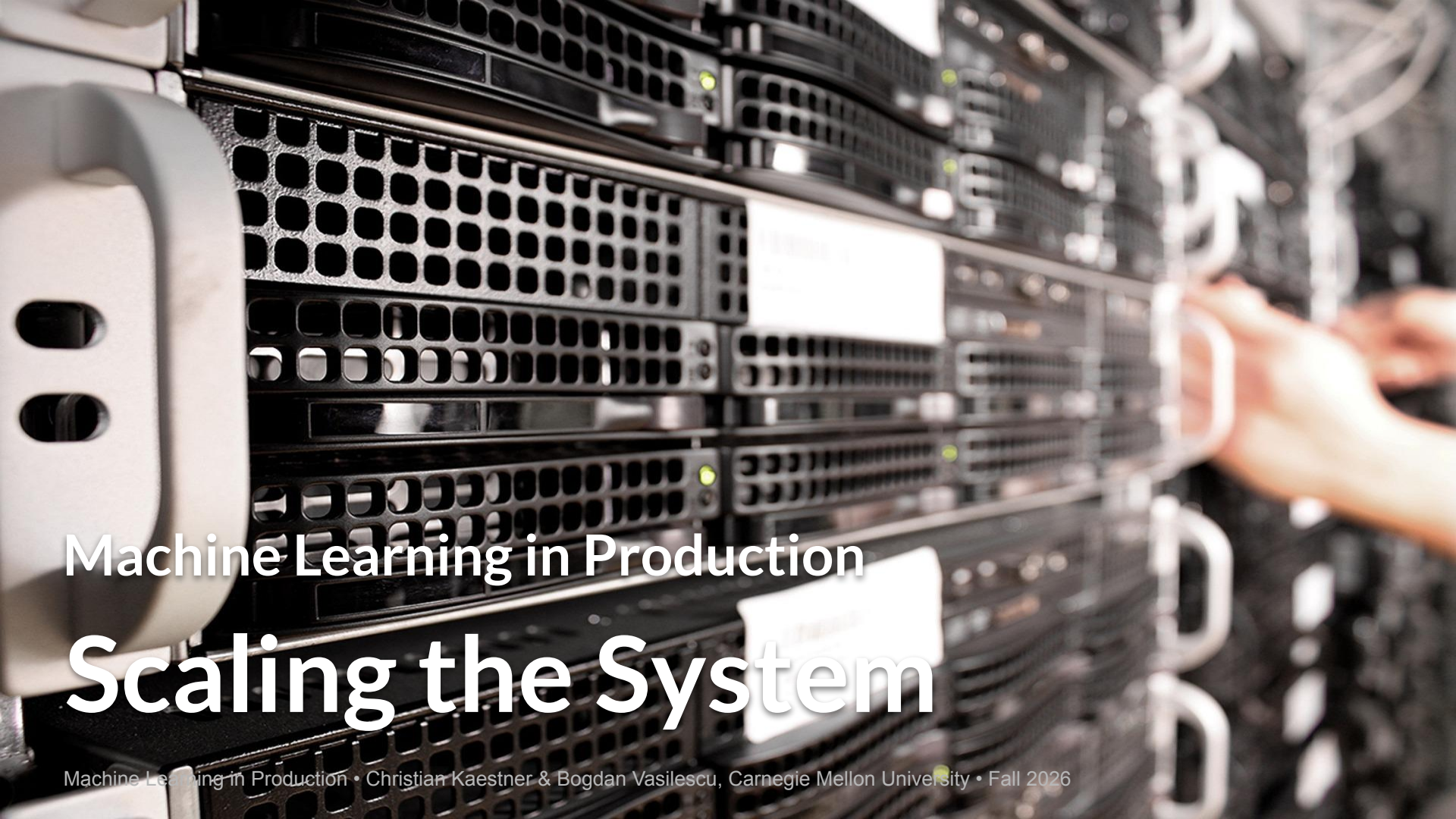




Machine Learning in Production Scaling the System

Administrativa

Midterm here during lecture time on Thursday (one week from now)

Old midterms on GitHub (link on website)

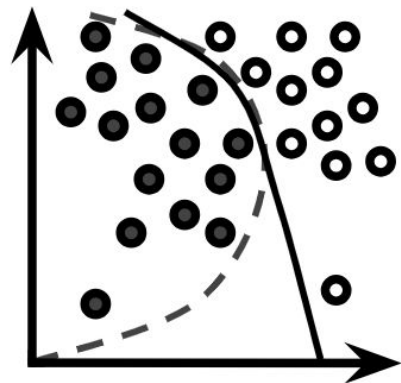
Grading rubric from last midterm posted on Slack

Consider `/breakout` to retry breakout feedback

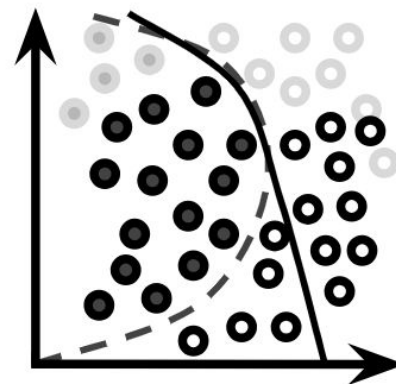
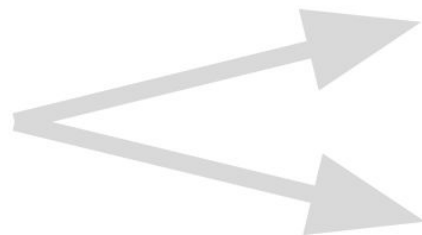
Notes allowed (6 sheets of papers, handwritten or printed, both sides)

Revisit: Drift and Data Cascades

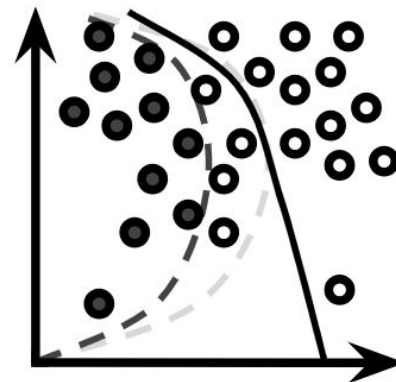
Data vs Concept Drift



Data drift:
changed distribution,
stable decision boundary,
inaccurate predictions



Concept drift:
changed decision boundary
stable distribution
inaccurate predictions



Legend:

- Data group A
- Data group B
- - - Real-world decision boundary
- Model's learned decision boundary

Drift & Model Decay

Concept drift (or concept shift)

- properties to predict change over time (e.g., what is credit card fraud)
- model has not learned the relevant concepts
- over time: different expected outputs for same inputs

Data drift (or covariate shift, virtual drift, distribution shift, or population drift)

- characteristics of input data changes (e.g., customers with face masks)
- input data differs from training data
- over time: predictions less confident, further from training data

Schema drift (upstream data changes)

- external changes in data pipeline (e.g., format changes in weather service)
- model interprets input data incorrectly
- over time: abrupt changes due to faulty inputs

How do we fix these drifts?

Breakout: Drift

Scenario: *Target inventory management, predicting how much inventory should be in stock next week*

As a group, post and follow up in #lecture, tagging group members and @mlipbot:

- *Concept Drift:*
- *Data Drift:*
- *Schema Drift:*

Data Cascades

"Compounding events causing negative, downstream effects from data issues, that result in technical debt over time."



Takeaway: Many data quality problems are not technical in nature, but due to bad assumptions, poor knowledge, or bad incentives

Common Data Cascades

Physical world brittleness

- Idealized data, ignoring realities and change of real-world data
- Static data, one time learning mindset, no planning for evolution

Inadequate domain expertise

- Not understand data and its context
- Involving experts only late for troubleshooting

Conflicting reward systems

- Missing incentives for data quality
- Not recognizing data quality importance, discard as technicality
- Missing data literacy with partners

Poor (cross-org.) documentation

- Conflicts at team/organization boundary
- Undetected drift

Sambasivan, N., et al. (2021). [“Everyone wants to do the model work, not the data work”: Data Cascades in High-Stakes AI.](#) In Proc. Conference on Human Factors in Computing Systems.

Today: Scaling Systems

Design and operations

Fundamentals of Engineering AI-Enabled Systems

Holistic system view: AI and non-AI components, pipelines, stakeholders, environment interactions, feedback loops

Requirements:

System and model goals
User requirements
Environment assumptions
Quality beyond accuracy
Measurement
Risk analysis
Planning for mistakes

Architecture + design:

Modeling tradeoffs
Deployment architecture
Data science pipelines
Telemetry, monitoring
Anticipating evolution
Big data processing
Human-AI design

Quality assurance:

Model testing
Data quality
QA automation
Testing in production
Infrastructure quality
Debugging

Operations:

Continuous deployment
Contin. experimentation
Configuration mgmt.
Monitoring
Versioning
Big data
DevOps, MLOps

Teams and process: Data science vs software eng. workflows, interdisciplinary teams, collaboration points, technical debt

Responsible AI Engineering

Provenance,
versioning,
reproducibility

Safety

Security and
privacy

Fairness

Interpretability
and explainability

Transparency
and trust

Ethics, governance, regulation, compliance, organizational culture

Suggested Readings

Nathan Marz. Big Data: Principles and best practices of scalable realtime data systems. Manning, 2015. Chapter 1: A new paradigm for Big Data

Martin Kleppmann. [Designing Data-Intensive Applications](#). OReilly. 2017.

Learning Goals

- Organize different data management solutions and their tradeoffs
- Understand the scalability challenges involved in large-scale systems, including machine learning
- Explain the tradeoffs between batch processing and stream processing and the lambda architecture
- Recommend and justify a design and corresponding technologies for a given system

Adding capacity



Stories of catastrophic success?

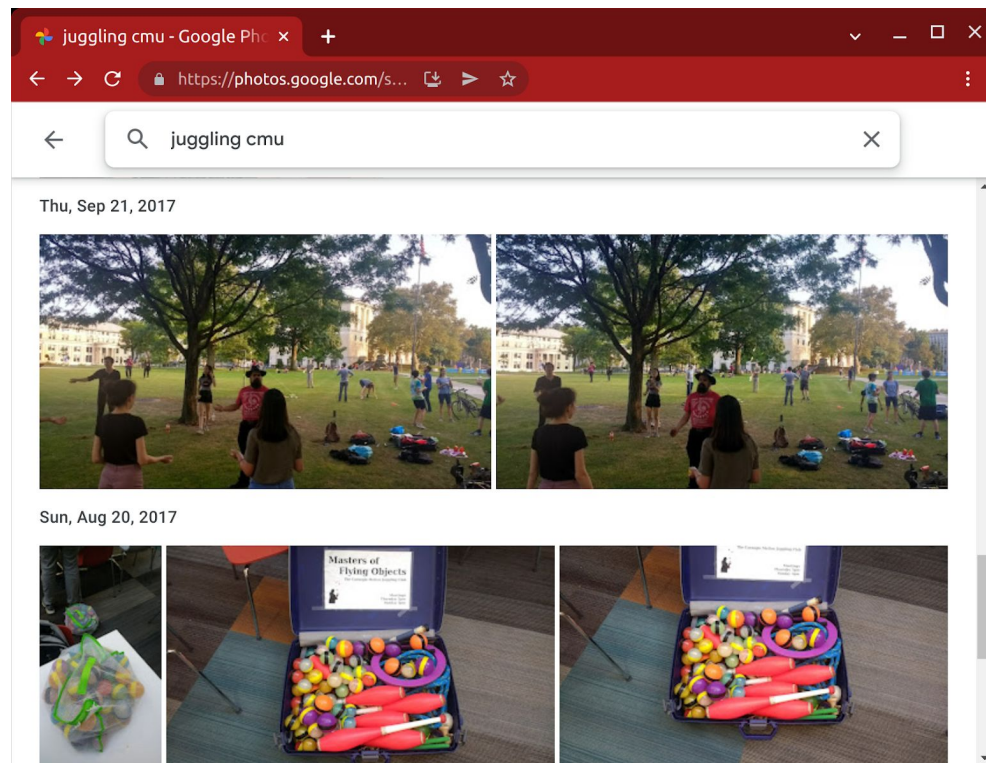
Scaling Strategies

Efficient Algorithms

Faster Machines

More Machines

Case Study



Code examples for today's concepts

Feel free to explore and follow along:

<https://github.com/mlip-cmu/scaling>

Data Management and Processing in ML-Enabled Systems

Kinds of Data

- Training data
- Input data
- Telemetry data
- (Models)

*all potentially with huge total volumes and high throughput
need strategies for storage and processing*

Data Management and Processing in ML-Enabled Systems

Store, clean, and update training data

Learning process reads training data, writes model

Prediction task (inference) on demand or precomputed

Individual requests (low/high volume) or large datasets?

Often both learning and inference data heavy, high volume tasks

Distributed Everything

Distributed data cleaning

Distributed feature extraction

Distributed learning

Distributed large prediction tasks

Incremental predictions

Distributed logging and telemetry

Distributed Systems and AI-Powered Systems

- Learning tasks can take substantial resources
- Datasets too large to fit on single machine
- Nontrivial inference time, many many users
- Large amounts of telemetry
- Experimentation at scale
- Models in safety critical parts
- Mobile computing, edge computing, cyber-physical systems

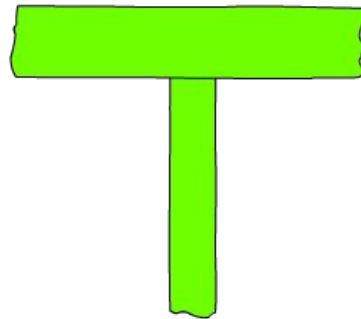
Reminder: T-Shaped People



"I-shaped"
Expert at one thing



Generalist
Capable in a lot of things
but not expert in any



"T-shaped"
Capable in a lot of things
and expert in one of them

Go deeper with: Martin Kleppmann. [Designing Data-Intensive Applications](#). O'Reilly. 2017.

Data Storage Basics

Relational Data Models

Photos:

photo_id	user_id	path	upload_date	size	camera_id	camera_setting
133422131	54351	/st/u211/1U6uFI47Fy.jpg	2021-12-03T09:18:32.124Z	5.7	663	f/1.8; 1/120; 4.44mm; ISO271
133422132	13221	/st/u11b/MFxlL1FY8V.jpg	2021-12-03T09:18:32.129Z	3.1	1844	f/2, 1/15, 3.64mm, ISO1250
133422133	54351	/st/x81/ITzhcSmv9s.jpg	2021-12-03T09:18:32.131Z	4.8	663	f/1.8; 1/120; 4.44mm; ISO48

Users:

user_id	account_name	photos_total	last_login
54351	ckaestne	5124	2021-12-08T12:27:48.497Z
13221	eva.burk	3	2021-12-21T01:51:54.713Z

Cameras:

camera_id	manufacturer	print_name
663	Google	Google Pixel 5
1844	Motorola	Motorola MotoG3

```
select p.photo_id, p.path, u.photos_total
from photos p, users u
where u.user_id=p.user_id and u.account_name = 'ckaestne'
```

Document Data Models

```
{
  "_id": 133422131,
  "path": "/st/u211/1U6uFl47Fy.jpg",
  "upload_date": "2021-12-03T09:18:32.124Z",
  "user": {
    "account_name": "ckaestne",
    "account_id": "a/54351"
  },
  "size": "5.7",
  "camera": {
    "manufacturer": "Google",
    "print_name": "Google Pixel 5",
    "settings": "f/1.8; 1/120; 4.44mm; ISO271"
  }
}
```

```
db.getCollection('photos').find( { "user.account_name": "ckaestne" })
```

Log files, unstructured data

```
127.0.0.1 - - [06/Dec/2025:02:49:12 +0000] "GET /img13.jpg HTTP/1.1" 200 48213
127.0.0.1 - - [06/Dec/2025:02:49:35 +0000] "GET /img27.jpg HTTP/1.1" 200 51877
127.0.0.1 - - [06/Dec/2025:03:52:36 +0000] "GET /main.css HTTP/1.1" 200 2310
127.0.0.1 - - [06/Dec/2025:04:17:03 +0000] "GET /img13.jpg HTTP/1.1" 200 48213
127.0.0.1 - - [06/Dec/2025:05:04:54 +0000] "GET /img34.jpg HTTP/1.1" 200 39012
127.0.0.1 - - [06/Dec/2025:05:38:07 +0000] "GET /img27.jpg HTTP/1.1" 200 51877
127.0.0.1 - - [06/Dec/2025:05:44:24 +0000] "GET /img13.jpg HTTP/1.1" 200 48213
127.0.0.1 - - [06/Dec/2025:06:08:19 +0000] "GET /img13.jpg HTTP/1.1" 200 48213
```

Tradeoffs



Data Encoding

Plain text (csv, logs)

Semi-structured, schema-free (JSON, XML)

Schema-based encoding (relational, Avro, ...)

Compact encodings (protobuf, ...)

Distributed Data Storage

Replication vs Partitioning

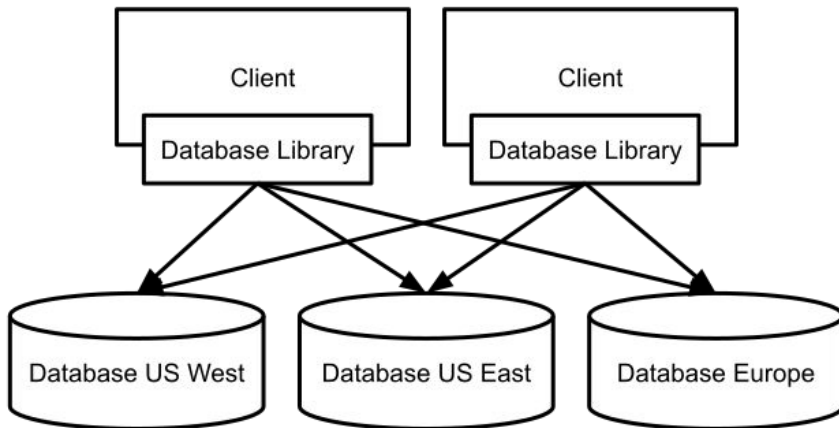


Partitioning

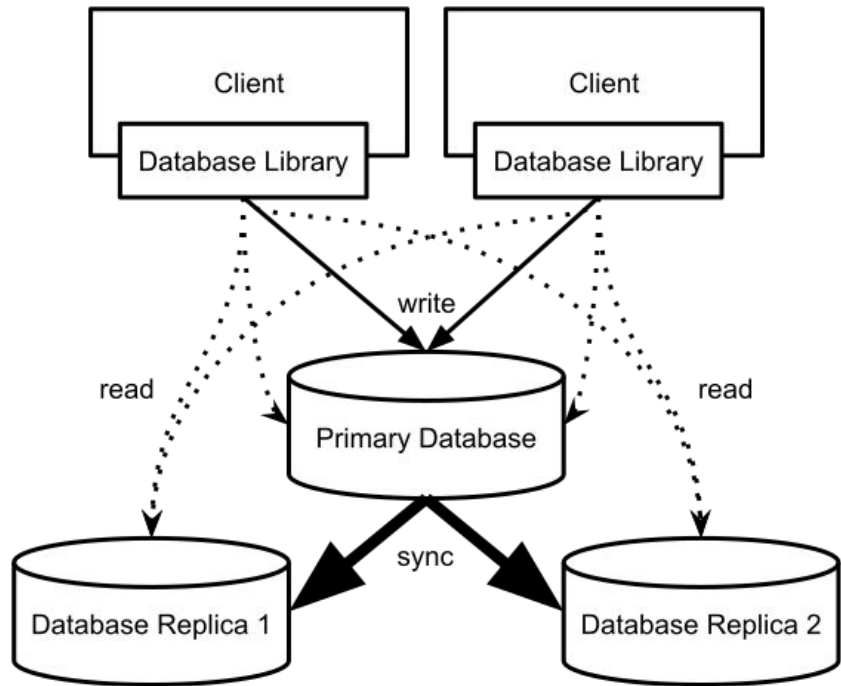
Divide data:

- *Horizontal partitioning*: Different rows in different tables; e.g., photos by date/user, hashing often used
- *Vertical partitioning*: Different columns in different tables; e.g., photo path and date vs. camera setting vs. tagged friends

Tradeoffs?



Replication with Leaders and Followers



Replication Strategies: Leaders and Followers

Write to leader, propagated synchronously or async.

Read from any follower

Elect new leader on leader outage; catchup on follower outage

Built in model of many databases (MySQL, MongoDB, ...)

Benefits and Drawbacks?

Example: Google File System

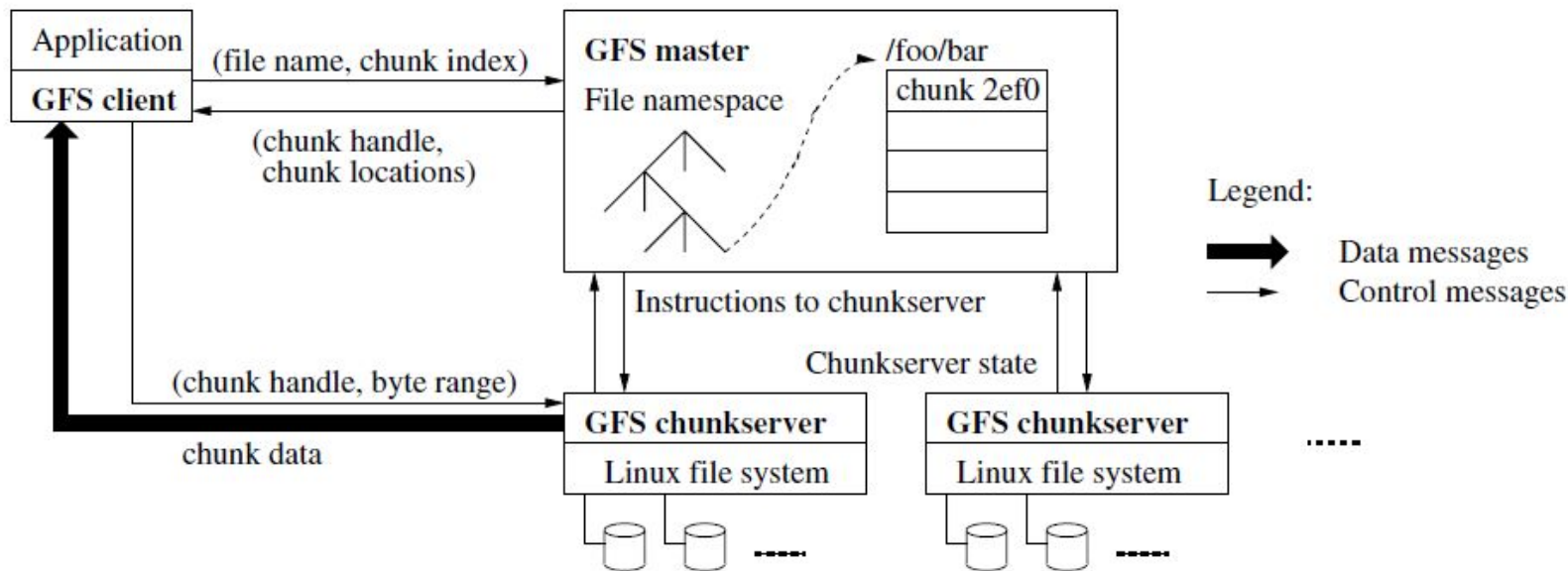


Figure 1: GFS Architecture

Beyond Leaders and Followers

Multi-Leader Replication

- Scale write access, add redundancy
- Requires coordination among leaders and resolution of write conflicts
- Offline leaders (e.g. apps), collaborative editing (like Google Docs, Automerge, Yjs), *EDB*, *Postgres Distributed*, ...

Leaderless Replication

- Client writes to multiple replica, propagate from there
- Versioning of entries (clock problem)
- Read from multiple replica (quorum required), version repair on reads/background repair process
- e.g. *Amazon Dynamo*, *Cassandra*, *ScyllaDB*

Transactions

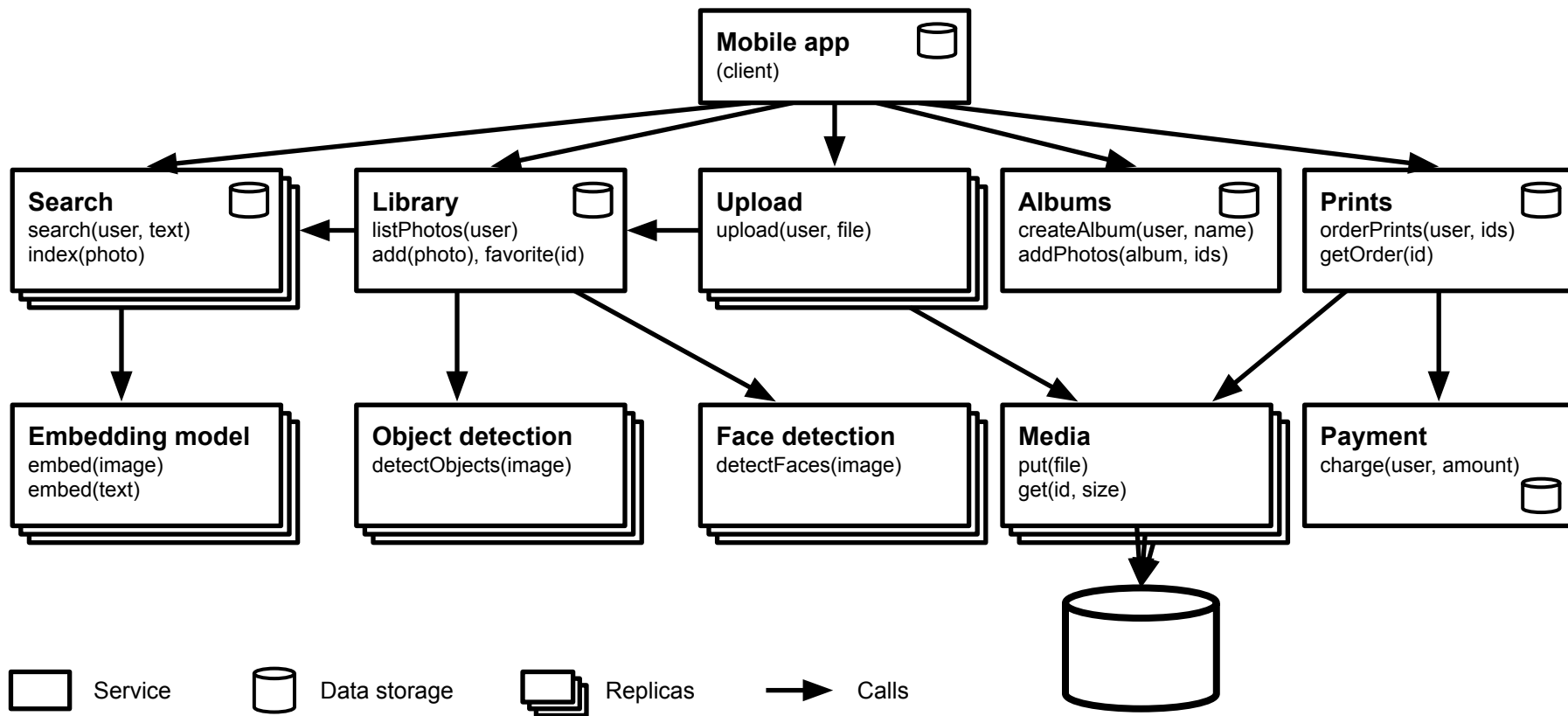
- Multiple operations conducted as one, all or nothing
- Avoids problems such as dirty reads, dirty writes
- Various strategies, including locking and optimistic+rollback
- Overhead in distributed setting

Data Processing (Overview)

- Services (online)
 - Responding to client requests as they come in
 - Evaluate: Response time
- Batch processing (offline)
 - Computations run on large amounts of data
 - Takes minutes to days; typically scheduled periodically
 - Evaluate: Throughput
- Stream processing (near real time)
 - Processes input events, not responding to requests
 - Shortly after events are issued

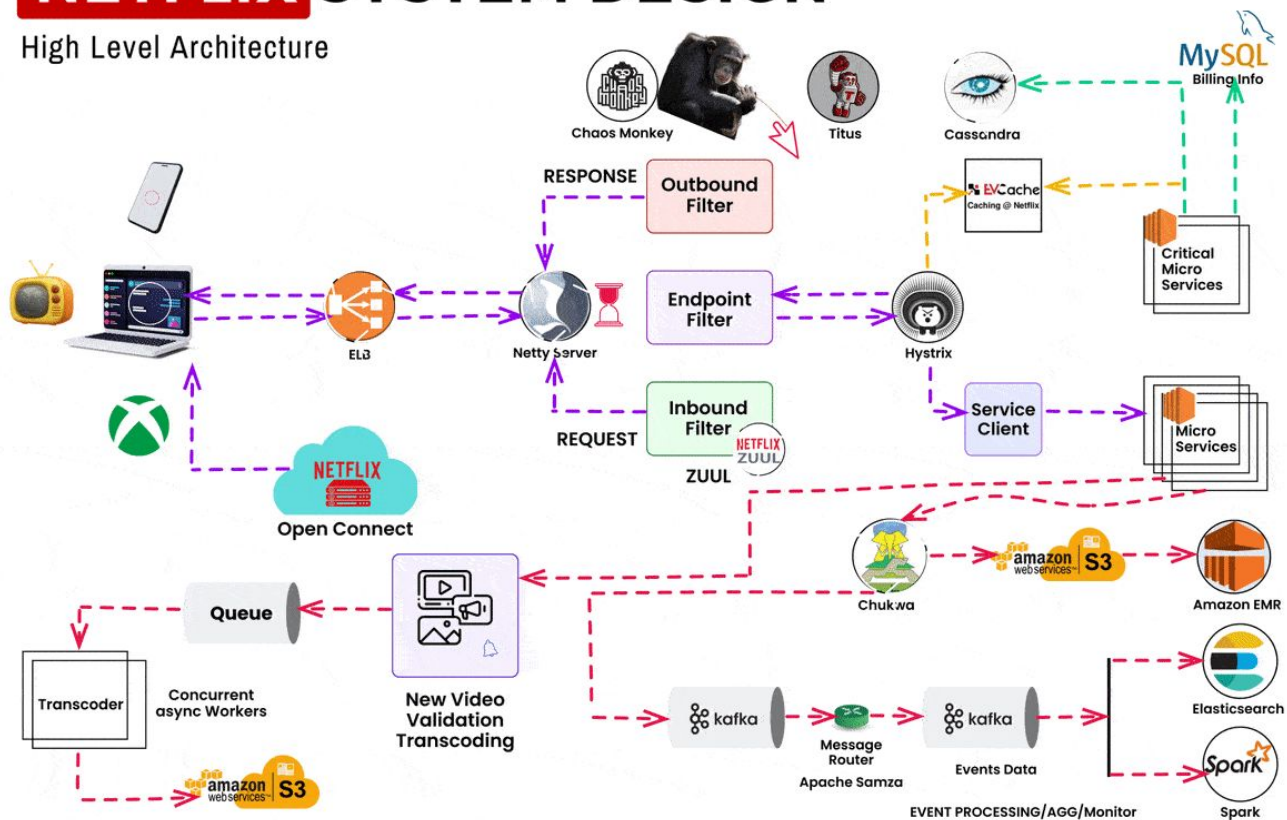
Microservices

Microservices



NETFLIX SYSTEM DESIGN

High Level Architecture



Microservices

Independent, cohesive services

- Each specialized for one task
- Each with own data storage
- Each independently scalable through multiple instances + load balancer

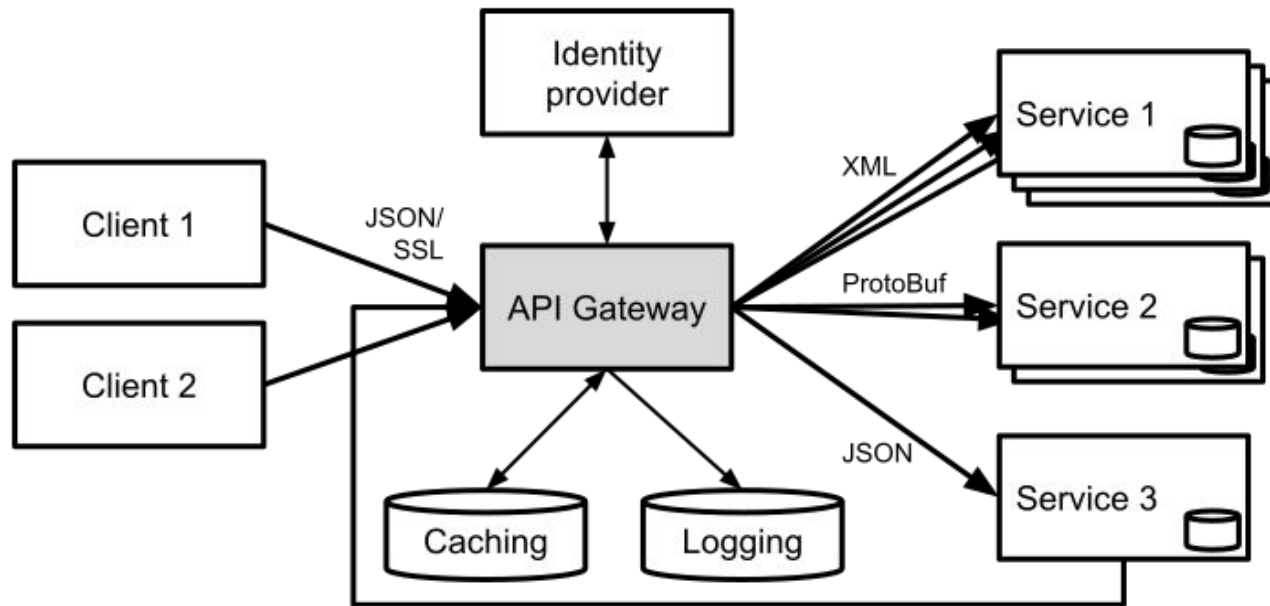
Remote procedure calls

Different teams can work on different services independently (even in different languages)

But: Substantial complexity from distributed system nature: various network failures, latency from remote calls, ...

Avoid microservice complexity unless really needed for scalability

API Gateway Pattern



Central entry point, authentication, routing, updates, ...

Batch Processing

Large Jobs

```
cat /var/log/nginx/access.log |  
  awk '{print $7}' |  
  sort |  
  uniq -c |  
  sort -r -n |  
  head -n 5
```

- Analyzing TB of data, typically distributed storage
- Filtering, sorting, aggregating
- Producing reports, models, ...

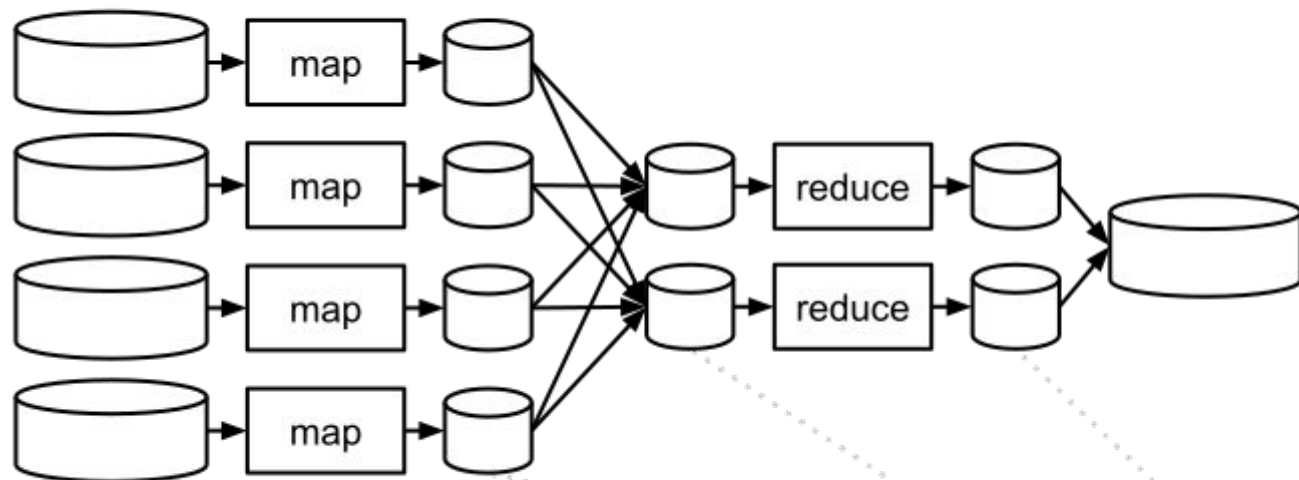
Partitioned
data storage

Map

Shuffle

Reduce

Result



```
02:49:12 127.0.0.1 GET /img13.jpg 200
02:49:35 127.0.0.1 GET /img27.jpg 200

03:52:36 127.0.0.1 GET /main.css 200
04:17:03 127.0.0.1 GET /img13.jpg 200

05:04:54 127.0.0.1 GET /img34.jpg 200
05:38:07 127.0.0.1 GET /img27.jpg 200

05:44:24 127.0.0.1 GET /img13.jpg 200
06:08:19 127.0.0.1 GET /img13.jpg 200
```

```
/img13, 1
/img27, 1

/img13, 1

/img34, 1
/img27, 1

/img13, 1
/img13, 1
```

```
/img13, 1
/img13, 1
/img13, 1
/img13, 1

/img27, 1
/img34, 1
/img27, 1
```

```
/img13, 4

/img27, 2
/img34, 1
```

Distributed Batch Processing

Process data locally at storage

Aggregate results as needed

Separate plumbing from job logic

MapReduce as common framework

MapReduce -- Functional Programming Style

Similar to shell commands: Immutable inputs, new outputs, avoid side effects

Jobs can be repeated (e.g., on crashes)

Easy rollback

Multiple jobs in parallel (e.g., experimentation)

Machine Learning and MapReduce



Dataflow Engines (Spark, Flink, Ray, ...)

Single job, rather than subjobs

More flexible than just map and reduce

Multiple stages with explicit dataflow between them

Often in-memory data

Plumbing and distribution logic separated

Key Design Principle: Data Locality

Moving Computation is Cheaper than Moving Data

-- [Hadoop Documentation](#)

Data often large and distributed, code small

Avoid transferring large amounts of data

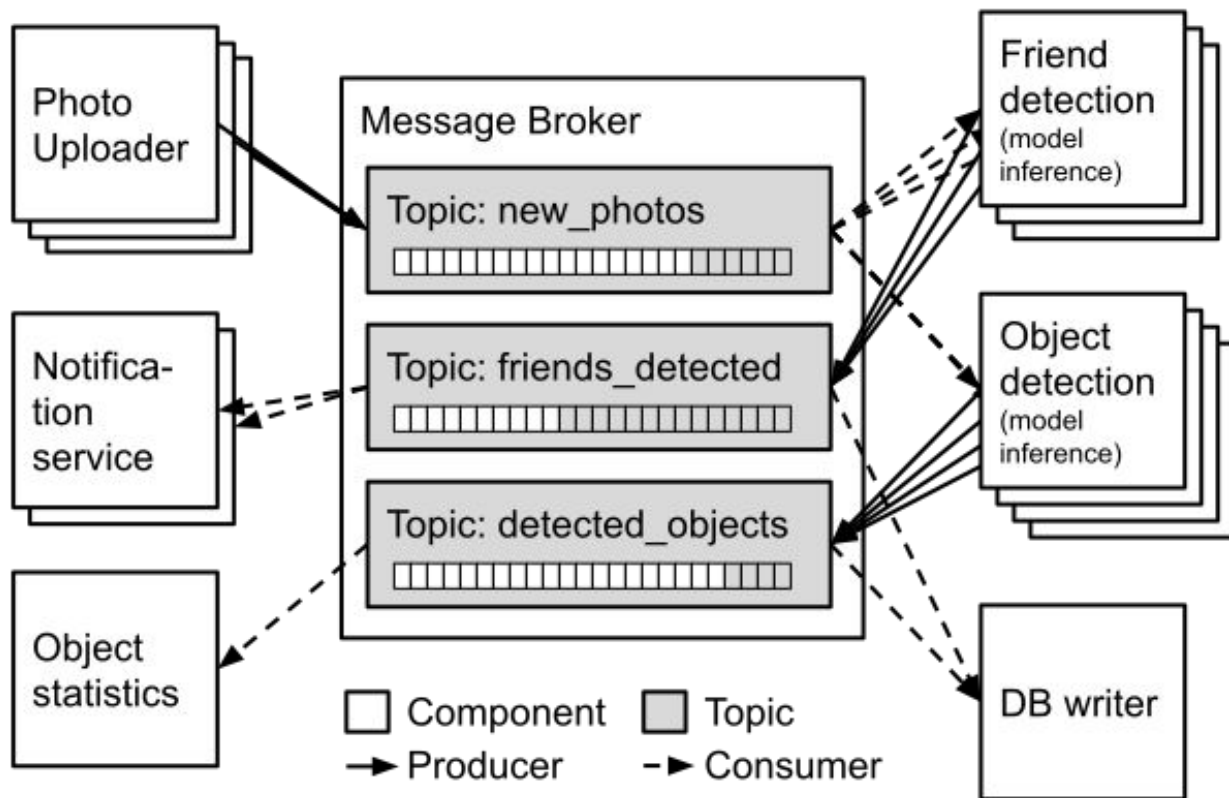
Perform computation where data is stored (distributed)

Transfer only results as needed

"The map reduce way"

Stream Processing

Stream Processing (e.g., Kafka)



Messaging Systems

Multiple producers send messages to topic

Multiple consumers can read messages

-> Decoupling of producers and consumers

Message buffering if producers faster than consumers

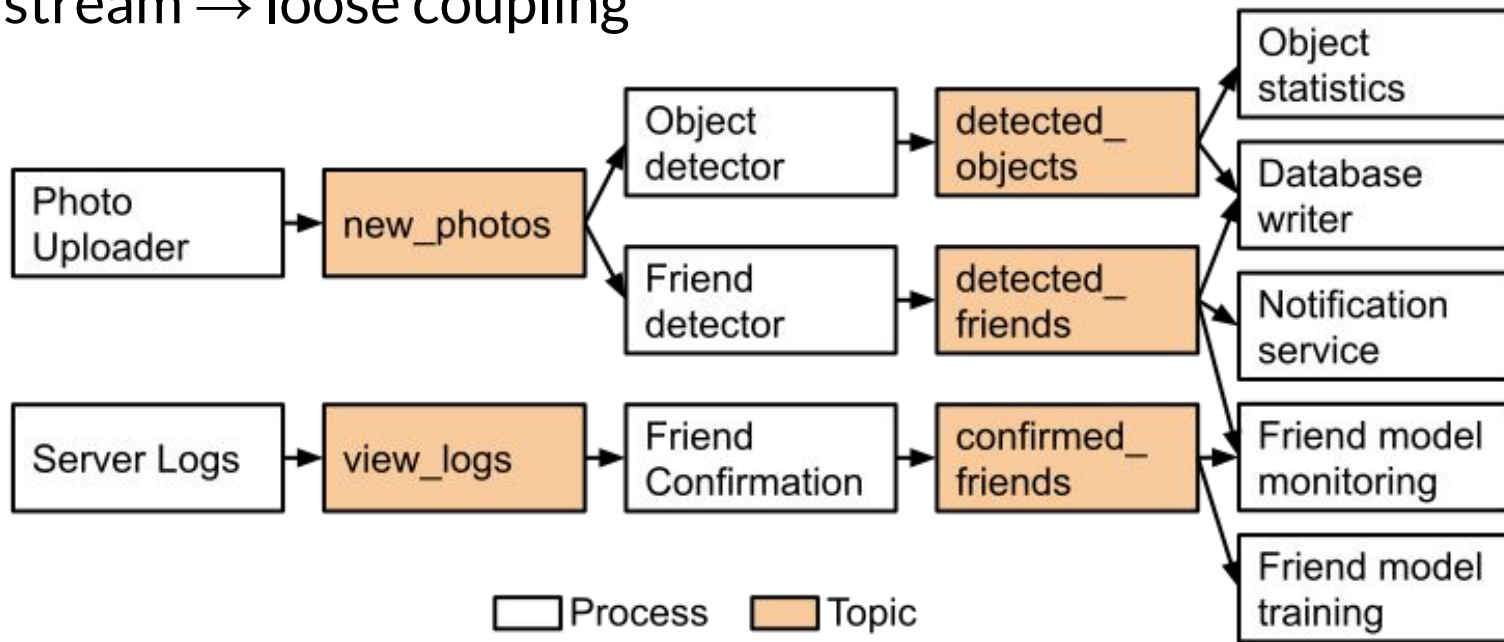
Typically some persistency to recover from failures

Messages removed after consumption or after timeout

Various error handling strategies (acknowledgements, redelivery, ...)

Common Designs

Like shell programs: Read from stream, produce output in other stream → loose coupling



Stream Queries

Processing one event at a time independently

vs incremental analysis over all messages up to that point

vs floating window analysis across recent messages

Works well with probabilistic analyses

Consumers

Multiple consumers share topic for scaling and load balancing

Multiple consumers read same message for different work

Partitioning possible

Design Questions

Message loss important? (enforce at-least-once processing)

Must messages never be processed repeatedly (enforce at-most-once processing)

Is the message order important?

Are messages still needed after they are consumed?

Stream Processing and AI-enabled Systems?



Reasoning about Dataflows

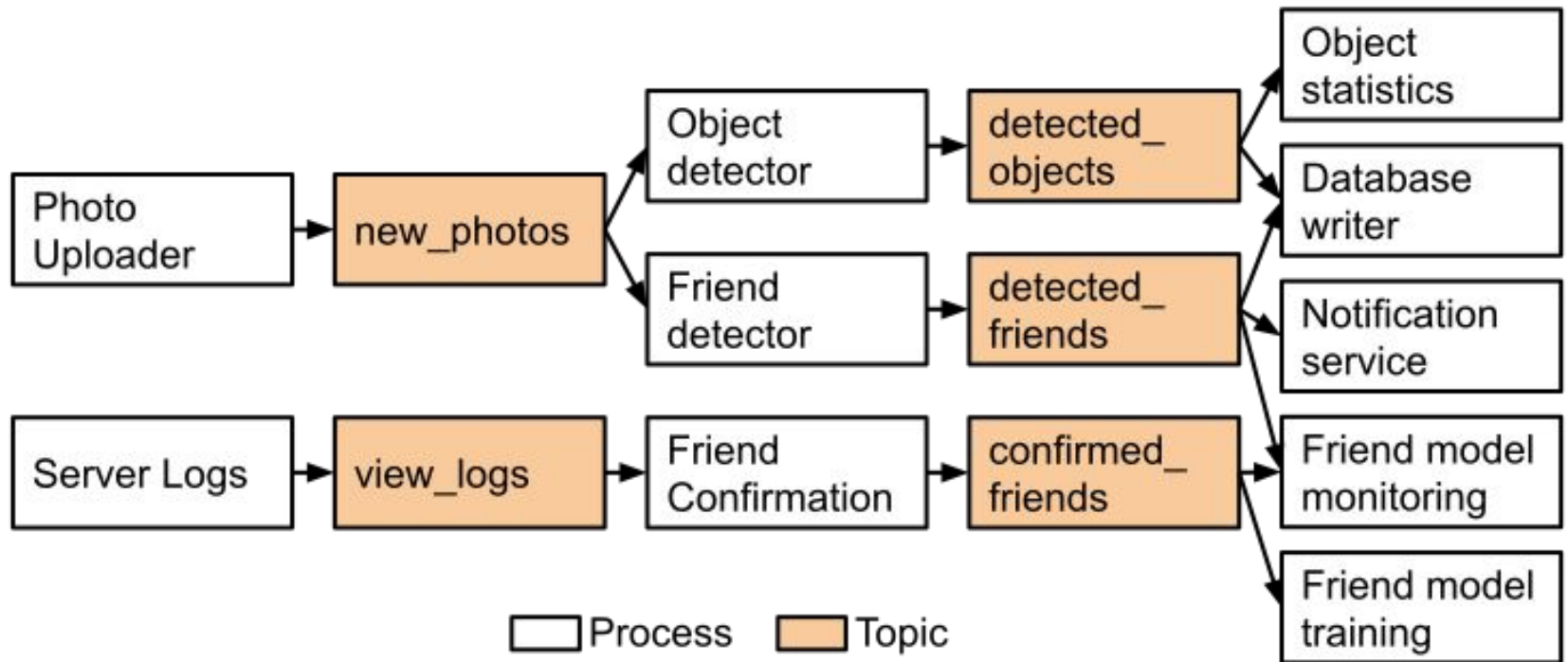
Many data sources, many outputs, many copies

Which data is derived from what other data and how?

Is it reproducible? Are old versions archived?

How do you get the right data to the right place in the right format?

Plan and document data flows



Event Sourcing

- Append only databases
- Record edit events, never mutate data
- Compute current state from all past events, can reconstruct old state
- For efficiency, take state snapshots
- *Similar to traditional database logs, but persistent*

```
addPhoto(id=133422131, user=54351, path="/st/u211/1U6uFl47Fy.jpg", date="2021-12-03T09:18:32.124Z")
updatePhotoData(id=133422131, user=54351, title="Sunset")
replacePhoto(id=133422131, user=54351, path="/st/x594/vipxBMFlLF.jpg", operation="/filter/palma")
deletePhoto(id=133422131, user=54351)
```

Benefits of Immutability (Event Sourcing)

- All history is stored, recoverable
- Versioning easy by storing id of latest record
- Can compute multiple views
- Compare *git*

On a shopping website, a customer may add an item to their cart and then remove it again. Although the second event cancels out the first event [...], it may be useful to know for analytics purposes that the customer was considering a particular item but then decided against it. Perhaps they will choose to buy it in the future, or perhaps they found a substitute. This information is recorded in an event log, but would be lost in a database [...].

Drawbacks of Immutable Data



The Lambda Architecture

3 Layer Storage Architecture

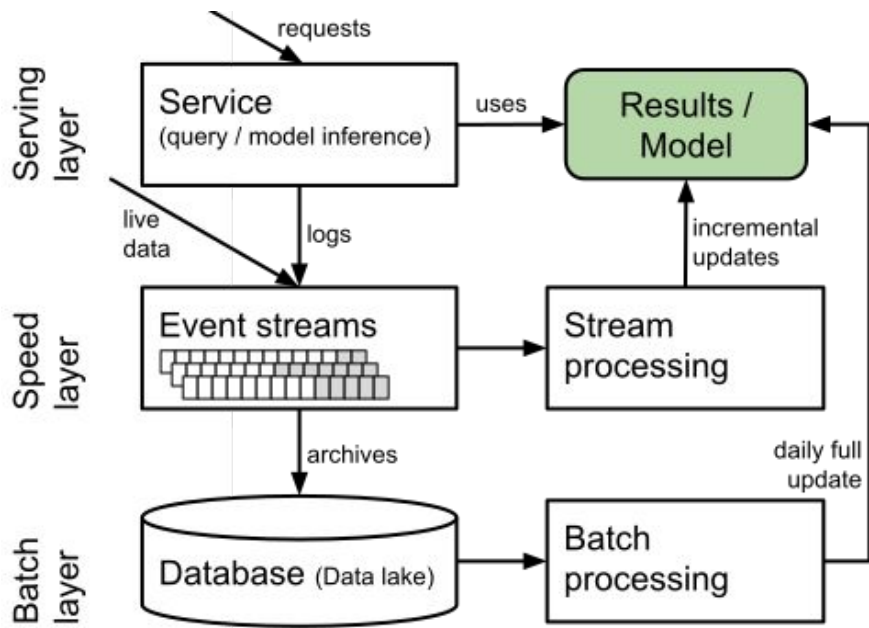
- Batch layer: best accuracy, all data, recompute periodically
- Speed layer: stream processing, incremental updates, possibly approximated
- Serving layer: provide results of batch and speed layers to clients

Assumes append-only data

Supports tasks with widely varying latency

Balance latency, throughput and fault tolerance

Lambda Architecture and Machine Learning



- Learn accurate model in batch job
- Learn incremental model in stream processor

Data Lake



Trend to store all events in raw form (no consistent schema)

May be useful later

Data storage is comparably cheap

Data Lake

Trend to store all events in raw form (no consistent schema)

May be useful later

Data storage is comparably cheap

Bet: *Yet unknown future value of data is greater than storage costs*

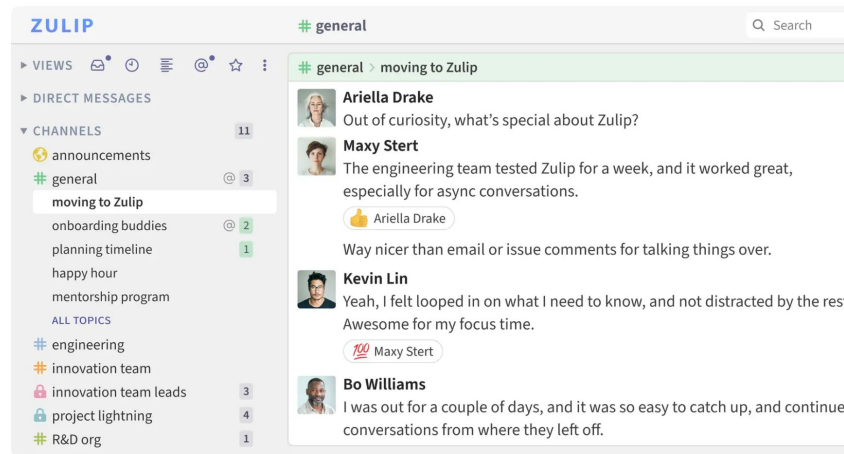
Lakehouse: open table formats (Apache Iceberg, Delta Lake) over Parquet files; query with Trino, Spark, or DuckDB

For logs: schema on read, e.g., Grafana Loki

Breakout: Scaling Zulip

Scenario: Scaling Zulip to a very large company (>50k users).

In addition to summarization (I1), need ML-based anomaly detection for compromised accounts (insider threats).



As a group, post and follow up in #lecture, tagging group members and @mlipbot:

- How to design scalable inference for summarization:
[service/stream/batch/lambda] [short justification]
- How to design scalable **training** of anomaly detection model:
[service/stream/batch/lambda] [short justification]

Excursion: ETL Tools

ETL: Extract, Transform, Load

- Transfer data between data sources, often OLTP -> OLAP system
- Many tools and pipelines
 - Extract data from multiple sources (logs, JSON, databases), snapshotting
 - Transform: cleaning, (de)normalization, transcoding, sorting, joining
 - Loading in batches into database, staging
- Automation, parallelization, reporting, data quality checking, monitoring, profiling, recovery
- Many commercial tools

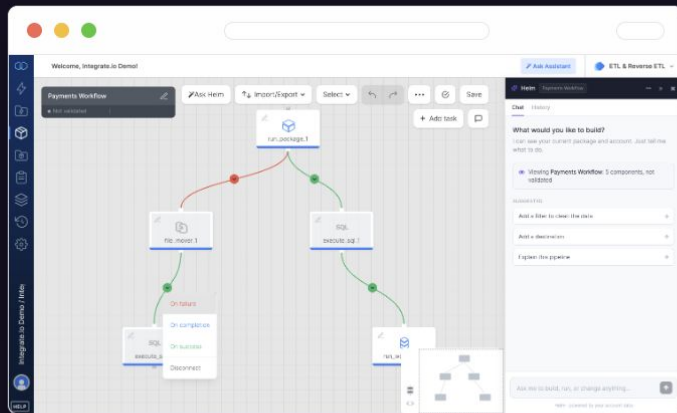
Examples of tools in [several lists](#)



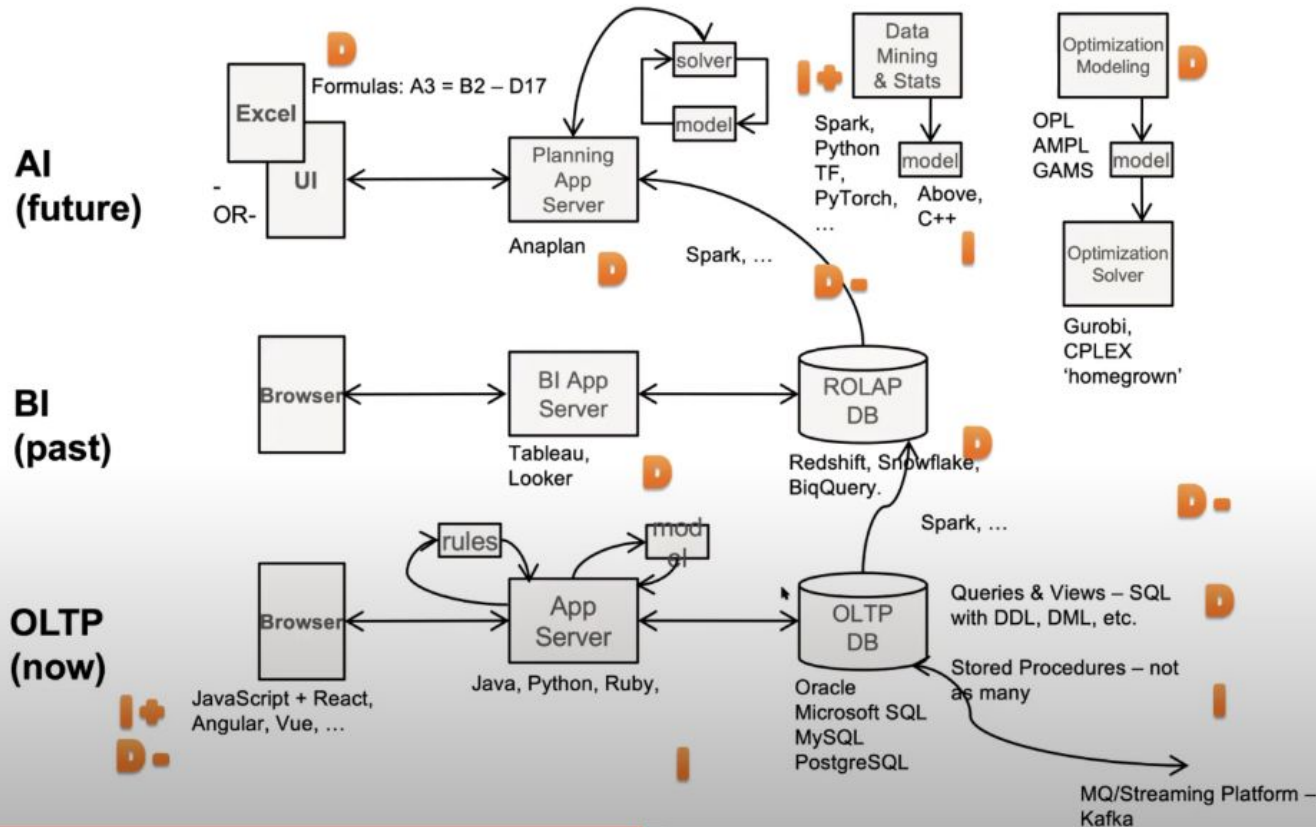
ETL pipelines your whole team can build and own.

Visual, drag-and-drop ETL with 150+ connectors and 220+ transformation steps, for one fixed fee. Business users build pipelines while IT stays in control.

→ Book a Demo



Enterprise Tech Stack – Now isn't much different



Molham Aref "[Business Systems with Machine Learning](#)"

Excursion: Performance Planning and Analysis

Performance Planning and Analysis

Ideally architectural planning upfront

- Identify key components and their interactions
- Estimate performance parameters
- Simulate system behavior (e.g., queuing theory)

Existing system: Analyze performance bottlenecks

- Profiling of individual components
- Performance testing (stress testing, load testing, etc)
- Performance monitoring of distributed systems

Performance Testing

Load testing: Assure handling of maximum expected load

Scalability testing: Test with increasing load

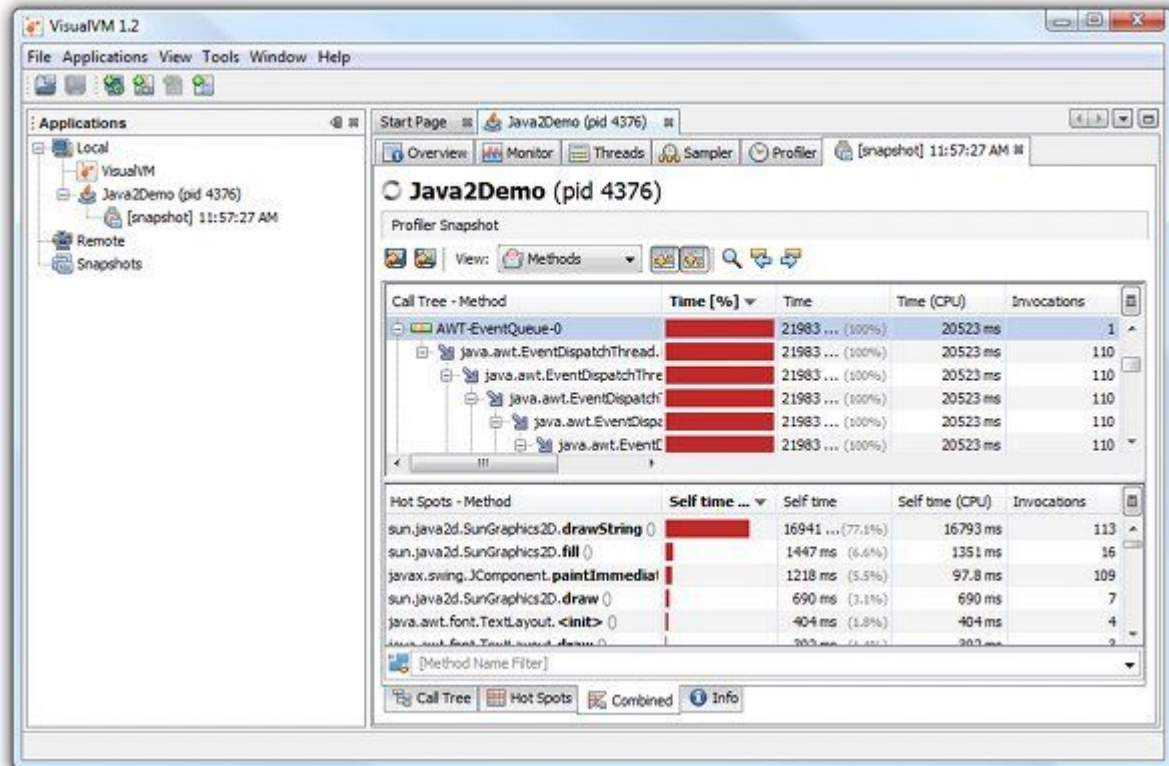
Soak/spike testing: Overload application for some time, observe stability

Stress testing: Overwhelm system resources, test graceful failure + recovery

Observe (1) latency, (2) throughput, (3) resource use

All automateable; tools like JMeter

Profiling



Performance Monitoring of Distr. Systems



Performance Monitoring of Distributed Systems

Instrumentation of (Service) APIs

Load of various servers

Typically measures: latency, traffic, errors, saturation

Tools: OpenTelemetry, Prometheus, Jaeger, Grafana

Monitoring long-term trends

Alerting

Automated releases/rollbacks

Canary testing and A/B testing

Summary

Large amounts of data (training, inference, telemetry, models)

Distributed storage and computation for scalability

Common design patterns (e.g., batch processing, stream processing, lambda architecture)

Design considerations: mutable vs immutable data

Distributed computing also in machine learning; many challenges through distribution: failures, debugging, performance, ...

Lots of tooling for data extraction, transformation, processing

Recommended reading: Martin Kleppmann. [Designing Data-Intensive Applications](#). O'Reilly. 2017.

Even More Recommended Readings

- Molham Aref "[Business Systems with Machine Learning](#)" Invited Talk 2020
- Sawadogo, Pegdwendé, and Jérôme Darmont. "[On data lake architectures and metadata management](#)." Journal of Intelligent Information Systems 56, no. 1 (2021): 97-120.
- Marz, Nathan, and James Warren. [Big Data: Principles and best practices of scalable realtime data systems](#). Manning, 2015.
- Smith, Jeffrey. [Machine Learning Systems: Designs that Scale](#). Manning, 2018.
- Polyzotis, Neoklis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. 2017. "[Data Management Challenges in Production Machine Learning](#)." In Proceedings of the 2017 ACM International Conference on Management of Data, 1723–26. ACM.
- Huyen, Chip. Designing Machine Learning Systems. O'Reilly, 2022.
- Reis, Joe, and Matt Housley. Fundamentals of Data Engineering. O'Reilly, 2022.